

# 1. Mesh generation

Reference codes are reachable at the folder “TC\_model\_generator”, to be opened as a project in your IDE.

## 1.1 Generation of the base configuration starting from CT images

You must use the script “Mesh\_generation.py”.

- **Lines 9 -13:** definition of the lists related to the single vertebrae to build, that is the lumbar ones first and then the thoracic ones (treated separately and then unified under a single mesh).  
Definition of the path containing the NIFTI file of the vertebral column. Images are reachable at the folder “TC\_images”. The considered image belongs to the serie number 25.  
If you want to change serie you can turn that number into one of the following ones: '00, 14, 20, 21, 27'.
- **Lines 16-27:** Reading of the Nifti 3D image, definition of a threshold to detect the first vertebra (equals to the maximum of labels to identify L1), and definition of a voxel size to detect little components related to disconnected regions for the next 3d mesh reconstruction.
- **Righe 29-53:** first loop for the reconstruction of lumbar vertebrae. Identification of a mask for the regions of the images which intensity is equal to the fixed threshold.  
In the three following lines there's the elimination of the disconnected regions which dimension is less than the voxel size.  
Then there's the reconstruction of the mesh of the considered region using the Marching Cubes algorithm. That mesh is properly regulated, applying light smoothing deleting noisy components, and computing triangle normals and its orientations.  
Finally, separator indexes are computed, that is the couples of indexes identifying the start and the end point of the specific vertebra, necessary for its later identification within the entire vertebral column. Updating of the total mesh of the lumbar vertebrae (progressively adding the meshes to the single vertebrae), the new start point, coincident to the next point of the last current point of the mesh, and the segmentation threshold (reduced by one to identify the next vertebra at the next iteration).
- **Lines 55-79:** the same procedure as the previous step but used for the thoracic vertebrae. The only difference is that a single mesh is built, containing all the 12 thoracic vertebrae. However, the separator indexes are computed again for the single vertebrae.
- **Lines 83-95:** Unification of the total mesh of the lumbar vertebrae and the one of the thoracic ones, forming the one of the entire vertebral column. Viewing of the obtained 3D mesh.
- **Line 104- :** Saving the meshes of the single lumbar vertebrae, of the torso and of the entire vertebral column. Those meshes will be saved in the folder “Dataset\Meshes\BasePose25” or “BasePoseNN”, considering a series number different from 25.  
In addition to that a text file containing the start and the end point for the single vertebrae (separators indexes) will be saved in the same folder.

## 1.2 Adaptation of the model

This step allows to identify a scale factor allowing the overlapping of the single meshes of the model obtained from CT images to the ones related to the OpenSim model. The reference code is “new\_spine.py”.

- **Lines 17 – 39:** loading of the point clouds related to the OpenSim model (reachable at the folder “OpenSim\_base\_pose”) and the ones related to the CT modello (reachable at the folder “Dataset\Base\_meshes\Model{serie number}”).
- **Lines 41- 51:** computing the scale factor as a ration between the mean distances of the points from the centroid of the point cloud, between the CT point cloud and the OpenSim ones. Final viewing of the enlarged OpenSim point cloud (according to the computed scale factor) and the CT one, proving the overlapping.
- **Lines 54-56:** Saving the scale factor in the “info” text file (previously used for the separator indexes of the single vertebrae).

## 2. Dataset Generation

This step allows to create different vertebral configurations, necessary to validate the next registration algorithms.

The reference folder is “Dataset\_processing”, to be opened in a new project within your IDE.

### 2.1 Generation of the moving point clouds

The reference code is “generate\_configuration.py”.

- **Lines 6-11:** definition of the fundamental parameters, that is the serie number related to the specific model obtained from CT and its related scale factor, and the labels (defining the three initial deformation degrees).
- **Lines 14-77:** defining the parameters useful to obtain the warped configuration starting from the base one: weigths related to the elementary rotations of the single vertebrae for each degree of freedom of the model, joints associated to each vertebra and their positions and orientations with respect to the same vertebra.
- **Lines 79-108:** definition of the functions to create the transform matrices that will be used to generate the moving point cloud.
- **Lines 111-158:** function generating the moving point cloud, starting from the fixed one. Identification of specific intervals regulating the values of the degrees of freedom according to the initial deformation classes (lables), giving back a random value within that interval. The function returns the values of the three degrees of freedom and the initial movement field between the fixed and the moving clouds.
- **Lines 160-162:** function for the voxel-based downsampling of the point clouds.
- **Line 165-194:** Generation and saving of the base downsampled point cloud. The external loop starts, iterating for each serie fo CT images. Definition of the folder containg the meshes of the single vertebrae of the base configuration, that is “Dataset\\Base\_meshes\\Model{serie number}”. There follows a first internal loop, reading the PLY files of the single base meshes within that folder. Once these clouds are obtained, they are aligned with the Open3D reference system, downsmapled by a 5 mm voxel size, in order to obtain clouds having a maximum point number of 10000 and unified to obtain the base cloud of the vertebral column. *(N.B.: to obtain the non-downsampled point clouds usable for the tests on the traditional algorithms, you can avoid the downsampling obtaining the original point clouds).*

- **Line 190-:** We are still in the main loop. Creation of an output folder, in this case "Dataset\\Clouds\_for\_leopard\\Model\_{serie number}". In this folder the unified base point cloud is saved as "base.ply".  
There are two next nested loops. The first one iterates on the initial deformation degree, and the second one generates 200 different configuration per each deformation degree. In the output folder (the same of the previous one) the moving point clouds are saved as "{deformation degree}{configuration number}" as well as a text file with the same name + "\_Info", containing the value of the 3 degrees of freedom of that configuration and the movement field between the fixed and the moving point cloud.  
For instance, the 18th moving configuration related to low initial deformations will be saved in the file "low17.ply" (remembering that the counter starts from 0) and its text file will be "low17\_Info.txt".

Once the code execution is finished, within the folder "Clouds\_for\_leopard", there will be some subfolders related to each CT serie, containing the fixed point clouds, the moving ones and their text files containing useful information.

## 2.2 Generation of the dataset for LeParD tests

The reference code is "generate\_dataset.py".

- **Lines 7-12:** definition of the fundamental parameters that allows the identification of the point clouds, that is the models and their scale factors and destination subgroups (of the dataset), and the deformation classes. Moreover the destination folder is defined as "Dataset\\4D\_Match\_dataset\_heavy\\split\\split".
- **Lines 14-20:** first external loop building the searching folder containing the point clouds per each vertebral column model. It is obtained as "Dataset\\Clouds\_for\_leopard\\Model" + {serie}", where {serie} represents the serie number of the considered model.  
The moving configuration is read then and the output folder is created in order to save the files related to the considered model.
- **Line 21-:** double loop iterating on the different deformation classes and on each configuration per class, reading the point cloud associated to the considered label and configuration. Conversion of the points of the fixed and moving clouds into an array and translation of both of them with respect to the centroid of the moving point cloud.  
Reading from the "Info" file, related to the moving cloud, of the moving field (linking the fixed to the moving point cloud) and saving it into a matrix. Checking that applying that field to the fixed point cloud you can obtain exactly the moving cloud.  
Next there's the computation of the rotation matrix and of the translation vector, respectively equals to the identity and the null vector, owing to the absence of global roto-translations.  
Finally, there's the computation of the correspondences, of the overlapping rate and of the metric index according to the previous cited functions.
- **Lines 55-66** are in charge of saving the computed information into a NPZ file, paying attention to save coherent information with the one of LeParD's original dataset.

Once the execution of the code is finished, there will be a folder named "4D\_Match\_dataset\_heavy\\split\\split" containing the training, validation and test folders, each of them containing subfolders related to their own model of the vertebral column, containing the NPZ files related to each couple of fixed and moving clouds.

## 2.3 Volume bounds computation

The reference code is "Compute\_yaml\_parameters".

The volume bounds essential to train the LeParD model are computed using the function at the **lines 11-45**. In particular, the minimum values along the three directions are initialized to  $+\infty$ , while the

maximum ones to  $-\infty$ . After that there are three nuzzled loops, the first one iterating on the different vertebral column models, the second one on each deformation class and the third one on each configuration. For each iteration the points of the source and the target point cloud are obtained, computing the minimum and the maximum value along each direction, considering both clouds. If the computed maximum value is greater than the current one, the new maximum value will be equal to the computed one. In the same way the absolute minimum value is obtained for each direction.

At **lines 56-58**, this function is called and the bounds are printed.

## 3. Registration algorithms

The project is at the folder “TC\_model\_registration”.

### 3.1 Rigid registration (FPFH + RANSAC + ICP)

The reference code is “Rigid\_ICP.py”.

The files used to test the algorithm are contained in the folder “Dataset//For\_tests//Model27”, which the base point cloud of the 27th CT serie and 25 different configurations per deformation degree are present. These clouds are not downsampled, so they contain the original points.

- **Lines 9-13:** definition of the main parameters: scale factors, labels and number of configurations per deformation degree (25).
- **Lines 16-30:** Function for the viewing of the point clouds at different moments of the registration (ignored during the tests).
- **Lines 33-49:** functions for the downsampling of the fixed and moving clouds, and for the extraction of the FPFH features.
- **Lines 52-65:** function for the execution of RANSAC. It returns the transform matrix computed by that algorithm.
- **Lines 68-73:** functions for the execution of the ICP algorithm. It returns the transform matrix computed by that algorithm.
- **Lines 76-121:** functions for the computation of the evaluation metrics: global registration error (global\_metrics), warping error (distance\_preservation\_error) and point-to-point metrics (distance\_metrics).
- **Lines 131-153:** definition of the paths of the fixed cloud (“Dataset\For\_tests\Model27\base.ply”) and the text file containing the separator indexes of each vertebra of the model (obtained at the paragraph 1.1 → “Dataset\Base\_meshes\Model27\Infos.tx”).

Definition of the paths of the text files which the results before and after the registration will be saved in, in the folder “Results” located in the same folder of the project. Writing of the header in those text files, in order to easily visualize the metrics and their values at the end of registration.

Saving of the separator indexes into a  $17 \times 2$  matrix, where the rows identify the ordered vertebrae and the columns the start and the end point per each of them. These indexes are directly imported from the text file generated at paragraph 1.1.

- **Lines 156 - :** Main loop, with an external loop iterating on the deformation classes, and an internal one iterating on the configurations per class (25), reading the corresponding moving cloud (generated at the paragraph 2.1).

Using those clouds the initial evaluation metrics are computed saving them into the text file “Pre.txt” in the folder “Results”, following the order expected by the header.

After that there’s a loop starting from unified point clouds (fixed and moving of the entire vertebral column), dividing them into the single lumbar vertebrae and the torso (unique group of thoracic vertebrae), saving each of them into a buffer (both for fixed and moving cloud).

After that there's a third nuzzled loop iterating on each fixed and moving clouds couple executing the rigid registration firstly downasmapling those clouds, then applying the RANSAC algorithm and finally the ICP refinement. The transform matrix obatained from this pipeline is computed e and applied on the fixed point cloud. The transformed point clouds are saved in the buffer

"src\_pc\_buffer\_icp", assumed as a reference for the final results evaluation.

At the end of this loop the global evaluation metric before and after registration is computed and printed.

Before computing the evaluation metrics there's a loop unifying the separetd clouds, because the metrics require a couple of unique point clouds.

Finally, the evaluation metrics are computed and saved in the text file "RANSAC+ICP.txt" in the folder "Results", respecting the order expected by the header.

## 3.2 CPD non-rigid registration

The script referred to is "NonRigid\_CPD.py".

- **Lines 11–190:** Functions analogous to those previously described for the RANSAC+ICP approach. The main differences are:

- **Lines 45–53:** Function for **point cloud normalization** (useful for facilitating the non-rigid registration process). It computes the point cloud centroid and a scaling factor, defined as the maximum dimension of the point cloud. The point cloud is then translated so that its centroid coincides with the origin of the global reference frame and is scaled by the computed factor. The function returns the normalized point cloud, the centroid (offset), and the scaling factor.
- **Lines 55–56:** Function for **point cloud denormalization**. It restores the point clouds to their original dimensions using the centroid and scaling factor obtained during the normalization stage.
- **Lines 84–150:** Function implementing the **optimized registration procedure**. It performs multiple registration iterations while dynamically adapting the algorithm parameters to the characteristics of the point clouds being aligned, ultimately returning the optimal registration result. As input, it requires the initial error (used to compute the algorithm parameters), the two point clouds to be aligned, and the denormalization parameters (centroid and scaling factor). The function returns the deformed point cloud resulting from the optimization process.
- **Lines 152–161:** Function for **deformation propagation**. It extends the deformation from the downsampled point cloud to all the points of the original point cloud by means of a linear interpolation based on inverse distance weighting (IDW), ensuring that neighboring points undergo similar displacements.

- **Lines 194–213:** Same procedure as in the RANSAC+ICP implementation. The paths are defined to load the reference point cloud and retrieve the separator indices (the same used for RANSAC+ICP). The path for saving the text file containing the registration results ("CPD.txt" in the "Results" folder) is also specified, the file header is written, and the separator indices are stored in an array.

- **Lines 217 -:** Registration loop. It consists of an outer loop iterating over the different deformation classes and an inner loop iterating over the different configurations within each class.

The path to the moving point cloud is defined, the fixed and moving point clouds are loaded, and the global registration error before alignment is evaluated and displayed.

At this point, the registration start time is recorded.

Both point clouds are then downsampled using a voxel-based approach (analogous to that used for rigid registration). A rigid pre-alignment using RANSAC is performed on the fixed point cloud whenever its initial displacement with respect to the moving point cloud exceeds 60 mm.

Subsequently, both downsampled point clouds are normalized, and their normalization centroid (offset) and scaling factor are computed. Once these operations are completed, the optimized registration procedure begins by calling the corresponding function (lines 84–150), providing as

input the initial global error, the normalized fixed and moving point clouds, and the centroid and scaling factor of the moving point cloud as denormalization parameters.

At the end of this process, the deformed fixed point cloud aligned with the downsampled moving point cloud is obtained. An empty point\_cloud object is then created and populated with a number of points exactly equal to that of the original fixed point cloud.

As the final step, the deformation is propagated by calling the corresponding function (lines 152–161), passing the points of the downsampled fixed point cloud, the downsampled deformed fixed point cloud, and the full-resolution deformed fixed point cloud as inputs.

Once this operation is completed, the current time is recorded as the registration end time, and the total registration time is computed as the elapsed interval between the registration start and end times.

Finally, the registration metrics are computed and saved to the text file in the order specified by the header written at the beginning of the script. At the end of the script execution, the **"Results"** folder will contain a text file named **"CPD.txt"**, storing all the registration results.

### 3.3 GMMReg non-rigid registration

The reference script is "GMMReg.py".

- **Lines 1–144:** The same functions used in the non-rigid registration workflow with CPD are employed. The only difference is the function between **lines 85–102**, where the four characteristic parameters of the algorithm are computed according to heuristic rules. The objective is to avoid optimization loops, which would significantly increase the algorithm's execution time.
- **Lines 148–168:** Same procedure as in the CPD registration workflow. First, the paths to the fixed point cloud and the corresponding text file containing the separator indices are defined. Next, the fixed point cloud is loaded. Then, the path to the text file where the registration results will be stored ("GMMReg.txt", located in the "Results" folder) is specified, and a header containing the names of the registration metrics to be evaluated is written to the file. Finally, the separator indices are read from the corresponding text file and stored in an array.
- **Lines 170 -:** Registration loop, entirely analogous to those previously described. As before, it consists of two nested loops: an outer loop iterating over each initial deformation class and an inner loop iterating over the different configurations within each class. The operations performed are identical to those described for the CPD registration workflow. The only differences are:
  - **Line 198:** The registration parameters are computed dynamically.
  - **Lines 200–203:** The registration\_gmmreg function from the l2\_dist\_regs module of the probreg library is called. The function receives as input the points of the normalized fixed point cloud, the normalized moving point cloud, and the registration parameters previously computed.

At the end of the script execution, a file named "GMMReg.txt" will be generated in the "Results" folder, containing the registration results.

### 3.4 SVR non-rigid registration

The reference script is "SVR.py".

- **Lines 1–148:** The same functions used in the CPD and GMMReg registration workflows are employed. The only difference is found in **lines 98–107**, where a function dynamically computes the two main registration parameters as a function of the initial global registration error.
- **Lines 152–172:** The same procedure described for the two previously presented non-rigid registration workflows is followed. The fixed point cloud is loaded, the path to the text file in which the registration results will be saved ("SVR.txt" in the "Results" folder) is defined, and a header specifying the registration metrics to be computed is written to the file. The separator indices are then read from the corresponding text file and stored in a  $17 \times 2$  array.

- **Line 175 onward:** Registration loop, identical to those used for the CPD and GMMReg registration workflows. The only difference is found in **lines 204–213**, where the registration parameters are first computed using the function defined in **lines 98–107**, based on the initial registration error. Subsequently, the `registration_svr` function from the `l2_dist_regs` module of the `proberg` library is called to perform the non-rigid registration.

## 4. Lepard data post-processing

The folder containing the data from Lepard is “Lepard\_post\_processing”.

### 4.1 Generation of mean-resolution transformed point clouds

The reference script is “`med_res_post_clouds_generator.py`”, and its purpose is to generate deformed point clouds from the transformations estimated by Lepard, with the same resolution as the point clouds contained in the dataset.

- **Lines 6–15:** Function for deformation propagation to higher-resolution point clouds. This is the same function used in the traditional non-rigid registration algorithms.
- **Lines 17–19:** Definition of the directories required for processing, namely the folder containing the medium-resolution source point clouds, the folder containing the registration results produced by the Lepard experiments, and the output folder where the deformed medium-resolution point clouds will be saved.
- **Lines 21–25:** Loading of the medium-resolution source point cloud.
- **Lines 27–:** Main processing loop. At each iteration, the prediction file generated by Lepard is loaded from the previously defined results directory. Each file follows the naming convention `prediction_{prediction_number}.npz`. The data stored in each file are then read.

First, the points of the source point cloud are extracted and used to construct the corresponding point cloud object. This point cloud is then translated by an offset equal to the distance between the centroids of the original source point cloud and the source point cloud reconstructed from the Lepard prediction.

- **Lines 41–51:** The transformations estimated by the model for the low-resolution point clouds are extracted and applied to the low-resolution source point cloud.
- Finally, these transformations are propagated to the medium-resolution source point cloud, producing the corresponding deformed medium-resolution point cloud.
- The resulting point cloud is saved in the output directory using the filename convention `prediction_{prediction_number}.ply`.

### 4.2 Conversion of the prediction-model correspondences

The reference script is “`NPZ_to_MAT.py`”, and its purpose is to convert the contents of an NPZ file into a MAT file, making the data easier to process within the MATLAB environment.

The input file is “`correspondences.npz`”, which contains a list associating each prediction generated by the Lepard testing framework with the corresponding model configuration used during the test. Each element of the list has the format “`{prediction_number}, {deformation_level}{configuration_number}`”.

The script reverses the order of the two elements in each entry of the list and saves the resulting data to a file named “`correspondences.mat`”.

## 4.3 N-ICP non-rigid alignment

The reference script is "N\_ICP\_refinement.m". It is a MATLAB file and therefore must be executed within the MATLAB environment.

- **First section (lines 1–6):** Workspace and command window cleanup, followed by loading of the cell array containing the prediction–configuration correspondences (previously converted from NPZ to MAT format).
- **Second section (lines 7–57):** Definition of the directory "Dataset\Lepard\_nr\_icp\_results", where the registration results will be saved, and initialization of the **times** cell array, which stores the execution time required by the algorithm for each prediction file.

This is followed by a loop iterating over each prediction–configuration pair, using as input the source point clouds (medium-resolution point clouds obtained in Section 4.1) and the target point clouds (target clouds used for the Leopard testing phase). Each point cloud is converted into a structure containing the fields:

- faces (left empty, as point clouds do not contain surface connectivity),
- vertices (coordinates of the point cloud),
- normals (point normals).

The registration options are then defined, including the possibility of using point normals, visualizing in real time the deformations applied to the source point cloud during registration, and using landmarks if available.

At line 47, the actual registration procedure is executed, returning the transformed source point cloud and the source-to-target transformation matrix.

Finally, the transformed point coordinates are saved in the output directory as files named "{deformation\_level}{configuration\_number}.mat". Lines 55–56 populate the times cell array.

- **Lines 59–60:** Execution times are saved into a file named "times.mat", stored in the output directory defined before the main loop

## 4.4 Extension of the deformations to high-resolution clouds

The reference script is "**High\_resolution\_post\_NICP.py**", and its purpose is to extend the transformations obtained in Section 4.3 to the original high-resolution point clouds.

- **Lines 7–19:** Definition of the main parameters (deformation level and number of configurations), and function for propagating deformations to higher-resolution point clouds.
- **Lines 22–25:** Loading of the high-resolution source point cloud.
- **Lines 27–30:** Loading of the medium-resolution source point cloud (used for the Leopard test).
- **Line 33:** Definition of the directory where the transformed high-resolution point clouds will be saved.
- **Lines 36 -:** Iteration loop over each deformation level and each configuration. Within this loop, the MAT file containing the transformed points associated with the current configuration is loaded, and a point cloud object at medium resolution is created from these points.

**Lines 49–58** call the function that propagates the transformation to the high-resolution source point cloud.

Finally, the resulting point cloud is saved in the output directory using a filename of the form "{deformation\_level}{configuration\_number}.ply".

## 4.5 Total execution time computation

The reference script is "**Total\_time.py**".

- **Lines 4–13:** Definition of the filename corresponding to the MAT file containing the execution times of N-ICP for the Leopard examples ("Leopard\_nr\_icp\_results\times\_nicp.mat"). The MAT file is loaded, the first column is read (containing the execution time values), and a dictionary is created that maps each Leopard prediction to the execution times required by N-ICP.
- **Lines 15–24:** Computation of the total execution time for each example. In particular, a loop iterates over each Leopard prediction, reading the corresponding files containing the execution times. These times are then summed with those required by N-ICP for the corresponding samples, and a dictionary is created associating each prediction with its total execution time.
- **Lines 26–34:** Loading of the NPZ file containing the correspondences between prediction numbers and the corresponding deformed configurations, followed by the creation of an associated dictionary. A loop then iterates over each prediction. Within this loop, the configuration associated with the current prediction is retrieved, along with its total execution time, and a dictionary is created linking each deformed configuration to its corresponding total execution time. **Note:** the dictionary is not ordered, meaning that the storage order of the values does not follow the labels [low, medium, high] + [0,...,29].
- **Lines 36 -:** Saving of the total execution times ("lep\_n\_icp\_times") using two nested loops that follow the ordering defined in the previous step. The resulting vector is saved into a file named "post\_nicp\_times.npz" in the "Leopard Tests" directory.

## 4.6 Registration metrics computation

The reference script is "**Lepard\_results\_generator.py**".

- **Lines 7–101:** Definition of the main parameters and of the functions required to compute the evaluation metrics.
- **Lines 104–110:** Definition of the directories containing the high-resolution source point clouds (obtained in the previous section) and the high-resolution target point clouds. Subsequently, the original (non-deformed) source point cloud is loaded.
- **Lines 113–123:** Loading of the separator indices corresponding to the individual vertebrae.
- **Lines 126–129:** Definition of the output file used to store the registration results ("**Lepard.txt**" in the "Results" folder), and writing of a header specifying the evaluation metrics used.
- **Lines 131 -:** Double loop in which the transformed source point clouds (obtained in the previous section) and the target point clouds are loaded. Given these point clouds, the registration metrics are computed using the appropriate functions and written to the previously defined text file, following the order specified in the header.

## Box plot of the final results

The reference script is "Box\_plots.py", located in the "TC\_model\_registration" folder.

- **Lines 7–18:** Definition of the main parameters (deformation levels and number of configurations per deformation level). Definition of the base directory containing the files with the registration results. Creation of a dictionary that maps each method to its corresponding text file containing the registration results (located in the base directory).
- **Lines 20–25:** Extraction of the evaluation metrics available for each method ("method\_metrics"), reading of the data from the corresponding text files, and conversion of these data into dictionaries containing matrices (data).
- **Lines 27–48:** Definition of important parameters for the subsequent plots, including the y-axis labels ("plot\_labels"), the scaling factors to be used in the plots ("plot\_scales"), and the figure titles associated with each metric ("metric\_labels").
- **Lines 50–101:** Definition of key parameters for the boxplots: list of metrics, list of methods, and list of colors associated with the boxes of each method.

This is followed by the main loop that generates a boxplot for each metric. Within this loop, for each deformation level and each method, a slice is computed for each level, assuming that each contains 30 different examples per metric. The index (idx) of the current metric in the corresponding method file is also stored. Subsequently, two vertical lines are drawn to separate the three deformation levels.

- **Line 72:** Boxplots for the considered metric are generated.
  - **Lines 73–84:** Text positioning settings are defined to insert titles identifying the deformation classes within the figure.
  - **Lines 85–87:** The boxes associated with the different methods are colored, and the median line within each box is also colored.
  - **Lines 88–97:** Settings related to the x-axis and y-axis titles (font and rotation) are applied, and the appropriate scaling is set for the plot.
  - **Lines 99–101:** The figure is saved as "BP-{metric}.jpg" in the same directory containing the result text files.
- **Lines 104 -:** Generation of boxplots for the execution time metric, following the same procedure described above (no loop over metrics is needed since only one metric is considered). The main difference is that the execution time for the "Lepard+N-ICP" method is not stored in the corresponding text file, but is instead read from the NPZ file obtained in Section 4.5.

- **Lines 106–110:** The contents of this NPZ file are loaded, and the "Time" metric is added to the list of available metrics for the Lepard+N-ICP method. Conversely, the execution time metric for the "Pre" method is set to None.